J2EE PRACTICAL EXAMINATION

8 Important Programs with Explanations and Complete Code

Topics covered

JDBC PreparedStatement CRUD • HTML GET Form • Servlet Calculator • doGet/doPost • Servlet + JDBC • JSP include/forward • JSP CRUD • application/session implicit objects

Study note: Examples use a MySQL database named `college`, an `employee` table, and a traditional Servlet/JSP project. Adapt database credentials and servlet URL mappings to your lab setup. For Tomcat 10+, use `jakarta.servlet.*`; for Tomcat 9 or older, use `javax.servlet.*`.

Common Database Setup

Run this SQL once in MySQL before practicing Programs 1, 5, and 7.

```
CREATE DATABASE college;
USE college;

CREATE TABLE employee (
    emp_no INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    designation VARCHAR(100),
    qualifications VARCHAR(150)
);
```

Optional shared connection helper (save as DBUtil.java; place it in a package accessible to your code).

```
import java.sql.*;

public class DBUtil {
    public static Connection getConnection() throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college";
        String user = "root";
        String password = "YOUR_MYSQL_PASSWORD"; // change this
        return DriverManager.getConnection(url, user, password);
    }
}
```

Ensure the MySQL Connector/J JAR is added to the project. In a web project, place the database utility class in WEB-INF/classes or the appropriate build output, and never expose real database passwords in public code.

1. JDBC CRUD Operations on Employee Table using PreparedStatement

CRUD means Create (INSERT), Read (SELECT), Update (UPDATE), and Delete (DELETE). This console program uses PreparedStatement parameters to safely pass values.

Setup / steps

• Create DBUtil.java using the common setup above.

• Run EmployeeCRUD.java. Insert a record first, then test display, update, and delete.

EmployeeCRUD.java

```
import java.sql.*;
import java.util.Scanner;

public class EmployeeCRUD {
    public static void main(String[] args) {
        try (Connection con = DBUtil.getConnection();
             Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1.Insert  2.Display  3.Update  4.Delete  5.Exit");
                System.out.print("Choice: ");
                int ch = Integer.parseInt(sc.nextLine());

                if (ch == 1) {
                    System.out.print("Employee No: ");
                    int no = Integer.parseInt(sc.nextLine());
                    System.out.print("Name: ");
                    String name = sc.nextLine();
                    System.out.print("Designation: ");
                    String des = sc.nextLine();
                    System.out.print("Qualifications: ");
                    String qual = sc.nextLine();

                    String sql = "INSERT INTO employee VALUES (?, ?, ?, ?)";
                    try (PreparedStatement ps = con.prepareStatement(sql)) {
                        ps.setInt(1, no);
                        ps.setString(2, name);
                        ps.setString(3, des);
                        ps.setString(4, qual);
                        System.out.println(ps.executeUpdate() + " row inserted.");
                    }
                } else if (ch == 2) {
                    String sql = "SELECT * FROM employee";
                    try (PreparedStatement ps = con.prepareStatement(sql);
                         ResultSet rs = ps.executeQuery()) {
                        while (rs.next()) {
                            System.out.println(rs.getInt("emp_no") + " | " +
                                rs.getString("emp_name") + " | " +
                                rs.getString("designation") + " | " +
                                rs.getString("qualifications"));
                        }
                    }
                } else if (ch == 3) {
                    System.out.print("Employee No to update: ");
                    int no = Integer.parseInt(sc.nextLine());
                    System.out.print("New designation: ");
                    String des = sc.nextLine();
                    String sql = "UPDATE employee SET designation=? WHERE emp_no=?";
                    try (PreparedStatement ps = con.prepareStatement(sql)) {
                        ps.setString(1, des);
                        ps.setInt(2, no);
                        System.out.println(ps.executeUpdate() + " row updated.");
                    }
                } else if (ch == 4) {
                    System.out.print("Employee No to delete: ");
                    int no = Integer.parseInt(sc.nextLine());
                    String sql = "DELETE FROM employee WHERE emp_no=?";
                    try (PreparedStatement ps = con.prepareStatement(sql)) {
                        ps.setInt(1, no);
                        System.out.println(ps.executeUpdate() + " row deleted.");
                    }
                } else if (ch == 5) {
                    break;
                } else {
                    System.out.println("Invalid choice.");
                }
            }
```

```
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
```

2. HTML Form using GET and Display Submitted Employee Information

The GET method sends form values in the URL query string. This example submits to a Servlet, which displays the values on the response page.

Setup / steps

• Save the HTML file in the web app root.

• Open employee-form.html through the server, not as a file:// URL.

• GET is suitable for non-sensitive retrieval/search values; do not send passwords or private data using GET.

employee-form.html

```
<!DOCTYPE html>
<html>
<head><title>Employee Form</title></head>
<body>
  <h2>Employee Details</h2>
  <form action="EmployeeGetServlet" method="get">
    Employee Number: <input type="number" name="empNo" required><br><br>
    Employee Name: <input type="text" name="empName" required><br><br>
    Designation: <input type="text" name="designation" required><br><br>
    Qualifications: <input type="text" name="qualifications" required><br><br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

EmployeeGetServlet.java (Tomcat 10+)

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/EmployeeGetServlet")
public class EmployeeGetServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        String no = request.getParameter("empNo");
        String name = request.getParameter("empName");
        String des = request.getParameter("designation");
        String qual = request.getParameter("qualifications");

        PrintWriter out = response.getWriter();
        out.println("<html><body><h2>Submitted Employee Details</h2>");
        out.println("<p>Employee Number: " + escape(no) + "</p>");
        out.println("<p>Employee Name: " + escape(name) + "</p>");
        out.println("<p>Designation: " + escape(des) + "</p>");
        out.println("<p>Qualifications: " + escape(qual) + "</p>");
        out.println("</body></html>");
    }

    private String escape(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("\"", "&quot;")
                .replace("'", "&#x27;");
    }
}
```

3. Servlet Calculator using Radio Buttons

The form lets the user choose Addition, Subtraction, Multiplication, or Division. The Servlet reads the selected operation and calculates the result.

Setup / steps

• Radio buttons share the same name op so only one operation can be selected.

• The Servlet handles division by zero and invalid numeric input.

calculator.html

```
<!DOCTYPE html>
<html>
<head><title>Servlet Calculator</title></head>
<body>
  <h2>Calculator</h2>
  <form action="CalculatorServlet" method="post">
    First number: <input type="number" step="any" name="a" required><br><br>
    Second number: <input type="number" step="any" name="b" required><br><br>
    <label><input type="radio" name="op" value="add" required> Addition</label>
    <label><input type="radio" name="op" value="sub"> Subtraction</label>
    <label><input type="radio" name="op" value="mul"> Multiplication</label>
    <label><input type="radio" name="op" value="div"> Division</label><br><br>
    <input type="submit" value="Calculate">
  </form>
</body>
</html>
```

CalculatorServlet.java (Tomcat 10+)

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/CalculatorServlet")
public class CalculatorServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();

        try {
            double a = Double.parseDouble(request.getParameter("a"));
            double b = Double.parseDouble(request.getParameter("b"));
            String op = request.getParameter("op");
            double result;

            switch (op) {
                case "add": result = a + b; break;
                case "sub": result = a - b; break;
                case "mul": result = a * b; break;
                case "div":
                    if (b == 0) {
                        out.println("<h3>Cannot divide by zero.</h3>");
                        return;
                    }
                    result = a / b;
                    break;
                default:
                    out.println("<h3>Invalid operation.</h3>");
                    return;
            }
            out.println("<h2>Result = " + result + "</h2>");
        } catch (NumberFormatException e) {
            out.println("<h3>Please enter valid numbers.</h3>");
        }
    }
}
```

4. Demonstrate doGet() and doPost() in a Servlet

doGet() handles HTTP GET requests; doPost() handles HTTP POST requests. This example uses two forms to show the difference.

Setup / steps

• GET parameters usually appear in the URL; POST form values are sent in the request body.

• POST is not automatically encrypted—use HTTPS for confidentiality.

get-post.html

```
<!DOCTYPE html>
<html>
<head><title>GET and POST Demo</title></head>
<body>
  <h2>GET Request</h2>
  <form action="GetPostServlet" method="get">
    Name: <input type="text" name="name" required>
    <input type="submit" value="Send with GET">
  </form>

  <h2>POST Request</h2>
  <form action="GetPostServlet" method="post">
    Name: <input type="text" name="name" required>
    <input type="submit" value="Send with POST">
  </form>
</body>
</html>
```

GetPostServlet.java (Tomcat 10+)

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/GetPostServlet")
public class GetPostServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        show(request, response, "doGet() was called");
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {
        show(request, response, "doPost() was called");
    }

    private void show(HttpServletRequest request,
                      HttpServletResponse response, String message)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        if (name == null) name = "";
        name = name.replace("&", "&amp;").replace("<", "&lt;")
                   .replace(">", "&gt;").replace("\"", "&quot;");
        out.println("<html><body><h2>" + message + "</h2>");
        out.println("<p>Hello, " + name + "</p></body></html>");
    }
}
```

5. Retrieve and Display Employee Records using Servlet and JDBC

This Servlet queries the employee table using JDBC and displays the rows in an HTML table.

Setup / steps

• Use DBUtil.java from the common setup.

• Visit /YourAppName/EmployeeListServlet after starting the server.

• Insert some rows using Program 1 before testing.

EmployeeListServlet.java (Tomcat 10+)

```
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/EmployeeListServlet")
public class EmployeeListServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(
                 "SELECT emp_no, emp_name, designation, qualifications FROM employee");
             ResultSet rs = ps.executeQuery();
             PrintWriter out = response.getWriter()) {

            out.println("<html><body><h2>Employee Records</h2><table border='1'>");
            out.println("<tr><th>No</th><th>Name</th><th>Designation</th><th>Qualifications</th></tr>");
            while (rs.next()) {
                out.println("<tr><td>" + rs.getInt("emp_no") + "</td><td>" +
                    esc(rs.getString("emp_name")) + "</td><td>" +
                    esc(rs.getString("designation")) + "</td><td>" +
                    esc(rs.getString("qualifications")) + "</td></tr>");
            }
            out.println("</table></body></html>");
        } catch (SQLException e) {
            throw new ServletException("Unable to retrieve employee records.", e);
        }
    }

    private String esc(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("\"", "&quot;")
                .replace("'", "&#x27;");
    }
}
```

6. JSP jsp:include and jsp:forward Actions

jsp:include includes another resource's output at request time. jsp:forward transfers request processing to another resource and does not continue normal output from the original JSP.

Setup / steps

• Place all JSP files in the same web application folder for this example.

• Test forward.jsp?role=admin and forward.jsp?role=student.

• The jsp:forward action should occur before the response is committed.

header.jsp

```
<h2>My College Website</h2>
<hr>
```

included.jsp

```
<p>This content is produced by included.jsp.</p>
```

welcome.jsp — demonstrates jsp:include

```
<%@ page contentType="text/html;charset=UTF-8" %>
<html><body>
  <jsp:include page="header.jsp" />
  <h3>Welcome Page</h3>
  <jsp:include page="included.jsp" />
  <p>This line appears after the included content.</p>
</body></html>
```

forward.jsp — demonstrates jsp:forward

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%
    String role = request.getParameter("role");
    if ("admin".equals(role)) {
%>
    <jsp:forward page="admin.jsp" />
<%
    } else {
%>
    <jsp:forward page="user.jsp" />
<%
    }
%>
```

admin.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<h2>Admin Page</h2>
<p>Forwarded to admin.jsp.</p>
```

user.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<h2>User Page</h2>
<p>Forwarded to user.jsp.</p>
```

7. JSP CRUD Operations using JDBC PreparedStatement

This single JSP demonstrates Insert, Display, Update, and Delete actions. It is suitable as a practical demonstration; in a production application, move JDBC logic into a DAO/service class and use MVC rather than putting database code in JSP.

Setup / steps

• Change YOUR_MYSQL_PASSWORD and ensure the MySQL Connector/J driver is available to the web application.

• For classroom use only: production apps should use POST-Redirect-GET, CSRF protection, validation, and a DAO/service layer.

• The JSP displays error text only for learning convenience; avoid exposing raw exception messages in deployed applications.

employee-crud.jsp

```
<%@ page import="java.sql.*" %>
<%@ page contentType="text/html;charset=UTF-8" %>
<%!
    Connection getCon() throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college";
        String user = "root";
        String password = "YOUR_MYSQL_PASSWORD";
        return DriverManager.getConnection(url, user, password);
    }
    String esc(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("\"", "&quot;")
                .replace("'", "&#x27;");
    }
%>
<%
    request.setCharacterEncoding("UTF-8");
    String action = request.getParameter("action");
    String message = "";

    if (action != null) {
        try (Connection con = getCon()) {
            if ("insert".equals(action)) {
                String sql = "INSERT INTO employee VALUES (?, ?, ?, ?)";
                try (PreparedStatement ps = con.prepareStatement(sql)) {
                    ps.setInt(1, Integer.parseInt(request.getParameter("empNo")));
                    ps.setString(2, request.getParameter("empName"));
                    ps.setString(3, request.getParameter("designation"));
                    ps.setString(4, request.getParameter("qualifications"));
                    ps.executeUpdate();
                    message = "Employee inserted.";
                }
            } else if ("update".equals(action)) {
                String sql = "UPDATE employee SET emp_name=?, designation=?, qualifications=? WHERE emp_no
                try (PreparedStatement ps = con.prepareStatement(sql)) {
                    ps.setString(1, request.getParameter("empName"));
                    ps.setString(2, request.getParameter("designation"));
                    ps.setString(3, request.getParameter("qualifications"));
                    ps.setInt(4, Integer.parseInt(request.getParameter("empNo")));
                    message = ps.executeUpdate() + " row(s) updated.";
                }
            } else if ("delete".equals(action)) {
                try (PreparedStatement ps = con.prepareStatement(
                        "DELETE FROM employee WHERE emp_no=?")) {
                    ps.setInt(1, Integer.parseInt(request.getParameter("empNo")));
                    message = ps.executeUpdate() + " row(s) deleted.";
                }
            }
        } catch (Exception e) {
            message = "Operation failed: " + e.getMessage();
        }
    }
%>
<!DOCTYPE html>
<html><head><title>Employee CRUD</title></head><body>
<h2>Employee CRUD using JSP + JDBC</h2>
<p><%= esc(message) %></p>

<h3>Insert Employee</h3>
<form method="post">
  <input type="hidden" name="action" value="insert">
  Employee No: <input type="number" name="empNo" required><br><br>
  Name: <input type="text" name="empName" required><br><br>
```

```
  Designation: <input type="text" name="designation"><br><br>
  Qualifications: <input type="text" name="qualifications"><br><br>
  <input type="submit" value="Insert">
</form>

<h3>Update Employee (all fields except number)</h3>
<form method="post">
  <input type="hidden" name="action" value="update">
  Employee No: <input type="number" name="empNo" required><br><br>
  New Name: <input type="text" name="empName" required><br><br>
  New Designation: <input type="text" name="designation"><br><br>
  New Qualifications: <input type="text" name="qualifications"><br><br>
  <input type="submit" value="Update">
</form>

<h3>Delete Employee</h3>
<form method="post">
  <input type="hidden" name="action" value="delete">
  Employee No: <input type="number" name="empNo" required>
  <input type="submit" value="Delete">
</form>

<h3>Employee Records</h3>
<table border="1" cellpadding="5">
<tr><th>No</th><th>Name</th><th>Designation</th><th>Qualifications</th></tr>
<%
    try (Connection con = getCon();
         PreparedStatement ps = con.prepareStatement(
             "SELECT emp_no, emp_name, designation, qualifications FROM employee");
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
%>
<tr>
<td><%= rs.getInt("emp_no") %></td>
<td><%= esc(rs.getString("emp_name")) %></td>
<td><%= esc(rs.getString("designation")) %></td>
<td><%= esc(rs.getString("qualifications")) %></td>
</tr>
<%
        }
    } catch (SQLException e) {
        out.println("<tr><td colspan='4'>Could not load records. Check database setup.</td></tr>");
    }
%>
</table>
</body></html>
```

8. JSP application and session Implicit Objects

application is a ServletContext object shared by the web application; session is an HttpSession object associated with a particular user session.

Setup / steps

• The application counter is shared across users, so concurrent access may require synchronization/atomic updates in more advanced examples.

• The session counter is separate for each session. A new browser session generally starts its own counter.

implicit-objects.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<html><body>
<h2>JSP Implicit Objects: application and session</h2>

<%
    Integer visits = (Integer) application.getAttribute("totalVisits");
    if (visits == null) visits = 0;
    visits = visits + 1;
    application.setAttribute("totalVisits", visits);

    Integer myVisits = (Integer) session.getAttribute("myVisits");
    if (myVisits == null) myVisits = 0;
    myVisits = myVisits + 1;
    session.setAttribute("myVisits", myVisits);
%>

<p>Application-wide page visits: <%= application.getAttribute("totalVisits") %></p>
<p>Visits in this user's session: <%= session.getAttribute("myVisits") %></p>
<p>Session ID: <%= session.getId() %></p>

<form method="post">
  <input type="submit" name="resetSession" value="Reset my session counter">
</form>
<%
    if (request.getParameter("resetSession") != null) {
        session.removeAttribute("myVisits");
        response.sendRedirect("implicit-objects.jsp");
        return;
    }
%>
</body></html>
```

Quick Practical Viva / Revision

What is JDBC? Java Database Connectivity; API used by Java applications to communicate with databases.

What is PreparedStatement? A precompiled SQL statement that accepts parameter values using placeholders (?) and helps prevent SQL injection when used correctly.

What is CRUD? Create, Read, Update, Delete.

What does doGet() do? Processes HTTP GET requests.

What does doPost() do? Processes HTTP POST requests.

What is a Servlet? A Java server-side component that handles requests and generates responses.

What is JSP? JavaServer Pages; a server-side technology for generating dynamic web content.

jsp:include vs jsp:forward? include adds another resource's output; forward transfers processing to another resource.

application vs session? application is shared across the web app; session stores data for one user session.

Why close JDBC resources? To release database connections, statements, and result sets; try-with-resources closes them automatically.

Compatibility reminder: Code shown with `jakarta.servlet.*` targets Tomcat 10+. For Tomcat 9 or older, replace the Servlet imports with `javax.servlet.*`. Use the servlet API supplied by the server; do not package the server's servlet API JAR inside your web app.